Milestone-02: Temporal Trend Analysis

Dhruthi Paritala

Table of Contents

1. Project Description	1
2. Dataset Used	1
3. Environment Setup	2
4. Data Exploration	3
5. Data Preprocessing 5.1. Image Loading 5.2. Ground Truth Extraction 5.3. Image Resizing & Downscaling 5.4. Density Map Generation 5.5. Tensor Conversion	4
6. Proposed System	6
7. Training Pipeline	7
8. Conclusion	8

1. Project Description

The goal of this milestone is to analyze the **temporal patterns of natural disaster declarations in the United States**. The analysis focuses on understanding how disasters evolve over time by examining yearly trends, disaster type variations, and seasonal patterns.

By applying data preprocessing and visualization techniques, meaningful insights can be derived from the dataset regarding disaster frequency and distribution. Temporal analysis helps identify long-term changes, seasonal effects, and peak disaster years.

Understanding these trends is important for **disaster preparedness, policy planning, and risk mitigation strategies**.

2. Dataset Used

The dataset used for this analysis contains records of **disaster declarations across different states in the United States**. Each record includes multiple attributes such as:

- Declaration Number
- Declaration Type
- Declaration Date
- State
- County
- Disaster Type
- Disaster Title
- Start Date
- End Date
- Assistance Programs

Among these fields, the **Declaration Date** column is the most important for temporal analysis. This column was converted into a datetime format to extract time-related features such as **year and month**.

These extracted attributes enable analysis of disaster occurrences over different time periods.

3. Environment Setup

The analysis was conducted using **Python in a Jupyter Notebook environment** within VS Code.

The following libraries were used:

- **Pandas** – For data loading and preprocessing
- **NumPy** – For numerical operations
- **Matplotlib** – For data visualization

A virtual environment was used to maintain dependency consistency and avoid package conflicts.

4. Data Exploration

Before performing temporal analysis, the dataset was explored to understand its structure and verify that the data was loaded correctly.

4.1 Loading the Dataset

The dataset was loaded using the **Pandas library** for further analysis.

4.2 Viewing Sample Data

The first few rows of the dataset were displayed using df.head() to understand the dataset structure.

4.3 Checking Columns and Data Types

Column names and data types were examined to ensure the dataset fields were correctly defined.

4.4 Inspecting Key Fields

Important columns such as **Declaration Date** and **Disaster Type** were reviewed as they are essential for temporal analysis.

5. Data Preprocessing

Data preprocessing was performed to prepare the dataset for accurate time-based analysis. Raw datasets may contain inconsistent formats, missing values, or incorrect entries that can affect the results. Therefore, several preprocessing steps were applied to clean and organize the dataset before performing temporal analysis.

5.1 Date Conversion

The **Declaration Date** column was converted from string format into **datetime format** using the Pandas to_datetime() function.

Steps:

1. Load the dataset using the Pandas library.
2. Identify the **Declaration Date** column in the dataset.
3. Convert the column into datetime format using pd.to_datetime().
4. Handle invalid date values using errors='coerce'.

Purpose:

- To ensure accurate interpretation of date values
- To enable time-based analysis of disaster data
- To allow extraction of important time features such as year and month
- To make grouping and sorting by date possible

5.2 Feature Extraction

After converting the date column, additional time-related features were extracted from the Declaration Date column.

Steps:

1. Extract the **Year** from the Declaration Date column.
2. Create a new column named **Year** in the dataset.
3. Extract the **Month** from the Declaration Date column.
4. Create a new column named **Month** in the dataset.

Purpose:

- To analyze how disasters change over time
- To identify yearly trends in disaster declarations
- To detect seasonal patterns within a year
- To support grouping and visualization of time-based data

5.3 Handling Missing Values

During preprocessing, the dataset was checked for missing or invalid values in the Declaration Date column.

Steps:

1. Check the dataset for missing values in the date column.
2. Identify rows with invalid or null date values.
3. Remove records containing missing or incorrect dates using dropna().
4. Verify the dataset after removing invalid records.

Purpose:

- To maintain data quality and consistency
- To prevent errors during data analysis
- To ensure reliable visualizations and results
- To avoid misleading interpretations caused by incorrect data

6. Proposed System

The proposed system analyzes temporal trends of US natural disaster declarations using data preprocessing and visualization techniques. The system processes the dataset to identify patterns such as yearly disaster trends, disaster type variations, and seasonal patterns.

6.1 Dataset Input

The disaster dataset is loaded into the Python environment using the Pandas library. The dataset contains information such as declaration date, disaster type, and location details.

6.2 Data Preprocessing

The Declaration Date column is converted into datetime format. From this column, two new features are extracted: Year and Month. Invalid or missing date values are removed to ensure accurate analysis.

6.3 Temporal Trend Analysis

The dataset is grouped by Year to analyze how the number of disasters changes over time. A line chart is used to visualize yearly disaster trends.

6.4 Disaster Type Analysis

The most frequent disaster types are identified and analyzed across different years. A multi-line chart is used to compare how different disaster types evolve over time.

6.5 Seasonal Analysis

Disasters are grouped by Month to study seasonal patterns. A bar chart is used to visualize the monthly distribution of disasters.

6.6 Insight Generation

Additional analysis such as cumulative disaster trends and peak disaster year detection is performed to identify important patterns in disaster occurrences

7. Training Pipeline

Features of the Pipeline

The implemented pipeline includes the following features:

- **Automated Data Processing** – The dataset is automatically cleaned and prepared for analysis.
- **Temporal Feature Extraction** – Year and Month are extracted from the declaration date to support time-based analysis.
- **Trend Visualization** – Multiple charts are created to visualize disaster patterns over time.
- **Insight Generation** – Important insi

Workflow of the Training Pipeline

The overall pipeline follows a structured sequence of steps:

1. **Load Dataset**
   Import the disaster dataset into the Python environment.
2. **Preprocess Data**
   Convert date columns and remove invalid records.
3. **Extract Time Features**
   Create Year and Month columns from the declaration date.
4. **Perform Temporal Analysis**
   Group data by year and disaster type to identify trends.
5. **Generate Visualizations**
   Create graphs such as line charts and bar charts.
6. **Extract Insights**
   Identify cumulative trends and peak disaster years.

8. Conclusion

In this milestone, temporal analysis of the US natural disaster dataset was successfully performed. The dataset was first preprocessed by converting the declaration date into datetime format and extracting important time-related features such as Year and Month. This preprocessing ensured that the dataset was suitable for time-based analysis.

Various visualizations were created to understand disaster trends over time. Yearly trend analysis helped identify how disaster declarations change across different years. Disaster type analysis allowed comparison of the most frequent disaster categories, while monthly analysis helped identify possible seasonal patterns in disaster occurrences.

Additionally, cumulative disaster trends and the peak disaster year were identified to highlight periods of high disaster activity. These insights help in understanding long-term disaster patterns and can support better disaster preparedness and planning.

Overall, the temporal analysis provided meaningful insights into disaster occurrences and demonstrated the importance of data visualization in understanding disaster trends.